

# Claude Code for Accounting Research

---

Module 1 — Foundations: Tools, Context, and the Research  
Map

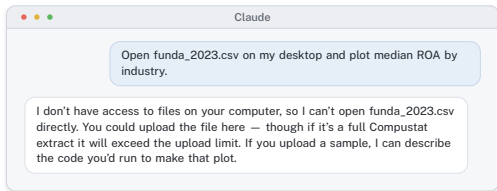
John Barrios

Yale School of Management

# Today's Arc

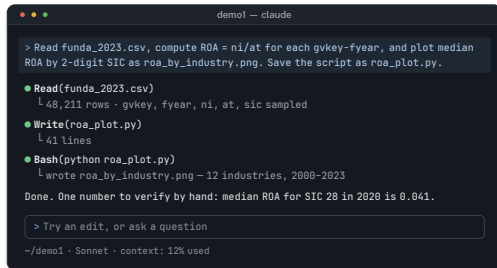
- **0:00–0:40** Lecture: tools, context, verification, research map
- **0:40–1:15** Live demo: one task, start to finish
- **1:15–1:25** Break
- **1:25–2:45** Lab: install, CLAUDE.md, one verified task
- **2:45–3:05** Debrief + exit ticket

# “AI Can’t Do Real Research Work”?



*Stylized session transcript (not a screenshot).*

Chat-level tool: no filesystem, no execution — rung 1 of the ladder.

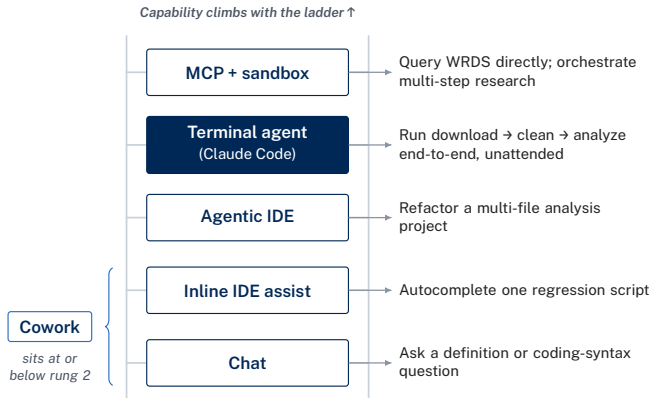


*Stylized session transcript (not a screenshot).*

Terminal agent: reads files, writes a named script, runs it locally — rung 4.

Same task, different rung. The limit was the tool you tried, not agentic AI.

# The Tool Ladder



Claude Code is on rung 4: it works in your terminal, reads your files, and runs code.

# Common Failure

## Dismissing AI after a Cowork-level disappointment

The tool did what it was built to do, and it was not built to touch your filesystem.  
Pick the rung the task needs.

# Claude Code as a Terminal-Resident RA

demo1 — claude

```
> Read funda_2023.csv, compute ROA = ni/at for each gvkey-fyear, and plot median ROA by 2-digit SIC as roa_by_industry.png. Save the script as roa_plot.py.
```

- Read(funda\_2023.csv)
  - └ 48,211 rows · gvkey, fyear, ni, at, sic sampled
- Write(roa\_plot.py)
  - └ 41 lines
- Bash(python roa\_plot.py)
  - └ wrote roa\_by\_industry.png — 12 industries, 2000-2023

Done. One number to verify by hand: median ROA for SIC 28 in 2020 is 0.041.

```
> Try an edit, or ask a question
```

~/demo1 · Sonnet · context: 12% used

1. **your prompt** — goal-first, names file, operation, aggregation, output

2. **tool calls** — the agent acting on your machine

3. **results it saw** — you can read everything it read

4. **handback** + a verification hook

5. **where you are:** directory, model, context used

*Stylized session transcript (not a screenshot). Anatomy of a Claude Code turn.*

It uses your own R/Python/Stata and your files. Manage it like an RA, not a search engine.

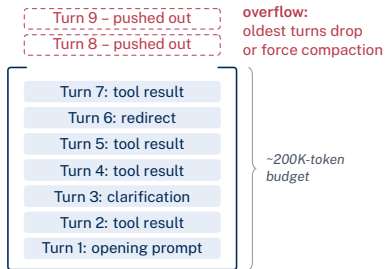
# Common Failure

Assuming the runtime is handled for you

It has no Python or R of its own; it uses whatever is installed locally. “Command not found” means check your environment, not the agent.

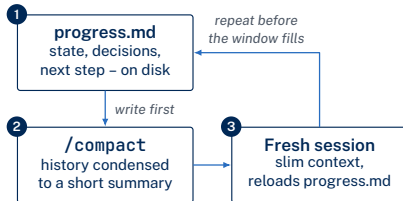
# Context Window Mechanics

(a) A finite window, re-sent every turn



Every message re-sends the full history – nothing is remembered independently.

(b) Intentional compaction



**Mantra 1 – Files persist; context doesn't.**

Write state to disk before it's gone.

Every turn re-sends the whole conversation.

Standard window: 200K tokens (some models: 1M); `/context` shows how full it is.

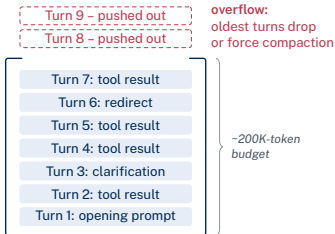
# Common Failure

## Importing your Dropbox mental model

Nothing persists across sessions unless it was written to a file a future session can read.

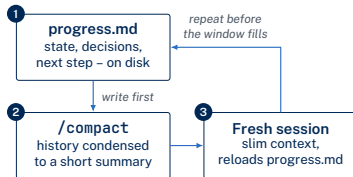
# Degradation and Intentional Compaction

(a) A finite window, re-sent every turn



Every message re-sends the full history –  
nothing is remembered independently.

(b) Intentional compaction



**Mantra 1 – Files persist; context doesn't.**

*Write state to disk before it's gone.*

In long sessions the window fills, and quality can drop before it is full.

## Mantra 1

Files persist; context doesn't.

# Common Failure

Treating compaction as forgetting everything

Compaction shortens the session; it does not erase it. Write the file, then compact.

# Trust, But Verify — The RA Standard

- Delegate freely; check the result before you build on it
- Same standard you apply to a real RA's work
- Pick one case you know well and check it by hand

## Verify this

Check one number against a source Claude did not touch before you trust the rest.

# Goal-First Prompting: Four Components

- **File** — what you're operating on
- **Operation** — the transformation to perform
- **Aggregation** — the level the result lives at
- **Output** — name and format, in advance

# Vague vs. Specific

## **Vague:**

*"Look at this Compustat file and tell me something about leverage."*

## **Specific:**

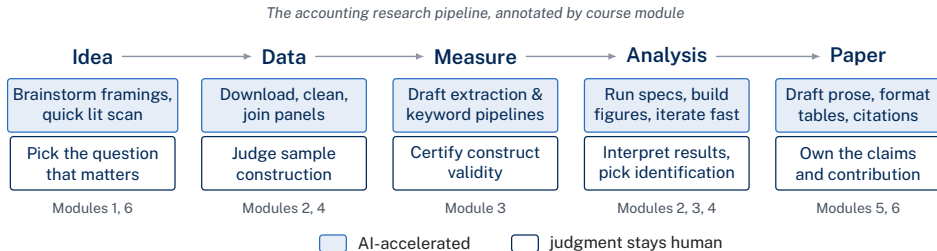
*"Read funda\_1990\_2023.csv. For each gvkey-fiscal year, compute book leverage as long-term debt / total assets, winsorize at 1st/99th pctile, mean by fiscal year x 2-digit SIC. Save leverage\_by\_industry\_year.csv and leverage\_trends.png."*

# Common Failure

## Describing the goal instead of the task

“Analyze this and tell me what’s interesting” hands the agent every decision. Before you hit enter, check for file, operation, aggregation, and output.

# The Accounting Research Workflow Map



idea → data → measure → analysis → paper

An agent speeds up some stages; the others stay with you.

# The Three Module 1 Mantras

Carry these through the course

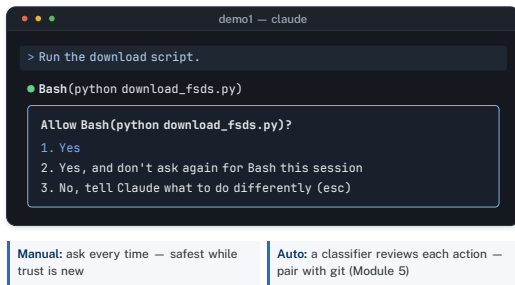
1. Files persist; context doesn't.
2. Trust, but verify — same as an RA.
3. Be specific: file, operation, aggregation, output.

They go on your verification protocol card. Module 6 returns to a harder version of the question.

# Next: Live Demo (0:40–1:15)

- One task, start to finish, in a fresh `~/demo1/`
- Launch, permission modes, goal-first prompt (SEC FSDS)
- Watch the loop; redirect, don't argue (Esc)
- Plan Mode preview; `CLAUDE.md`; verify the count

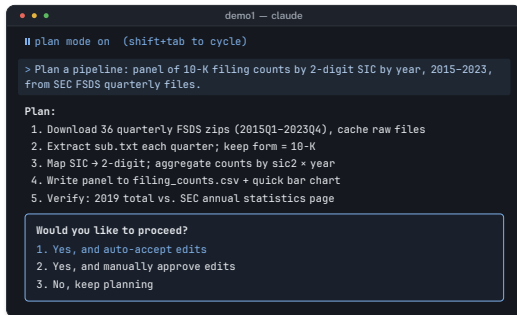
# What You'll See: The Permission Ask



*Stylized session transcript (not a screenshot).*

**Manual:** asks before every file edit, command, or network call.  
**Auto:** a classifier model passes routine actions, blocks risky ones.  
New sessions start in Auto; Shift+Tab to change.

# What You'll See: Plan Mode



```
demo1 — claude

|| plan mode on (shift+tab to cycle)

> Plan a pipeline: panel of 10-K filing counts by 2-digit SIC by year, 2015-2023,
from SEC FSDS quarterly files.

Plan:
1. Download 36 quarterly FSDS zips (2015Q1-2023Q4), cache raw files
2. Extract sub.txt each quarter; keep form = 10-K
3. Map SIC → 2-digit; aggregate counts by sic2 × year
4. Write panel to filing_counts.csv + quick bar chart
5. Verify: 2019 total vs. SEC annual statistics page

Would you like to proceed?
1. Yes, and auto-accept edits
2. Yes, and manually approve edits
3. No, keep planning
```

*Stylized session transcript (not a screenshot).*

Design before code: review the plan, then execute — or don't.

Shift+Tab cycles Manual → accept-edits → Plan Mode; the status bar shows which.

# Next: The Lab (1:25–2:45)

- **Part A (25 min):** environment — `claude` launches, `~/lab1/`
- **Part B (15 min):** write `CLAUDE.md`, restart, confirm it loads
- **Part C (40 min):** tiered task — base / standard / stretch
- Verify checklist: one output checked against an outside source

# Debrief, Exit Ticket, and Homework

- Debrief: what broke? Esc or argue? Whose prompt was vaguer than expected?
- Exit ticket: what clicked, what broke, what to verify next time
- **Homework:** confirm your WRDS account + Duo enrollment; test `ssh wrds` before Module 2